

架构设计文档

1. 架构概览（含架构图）

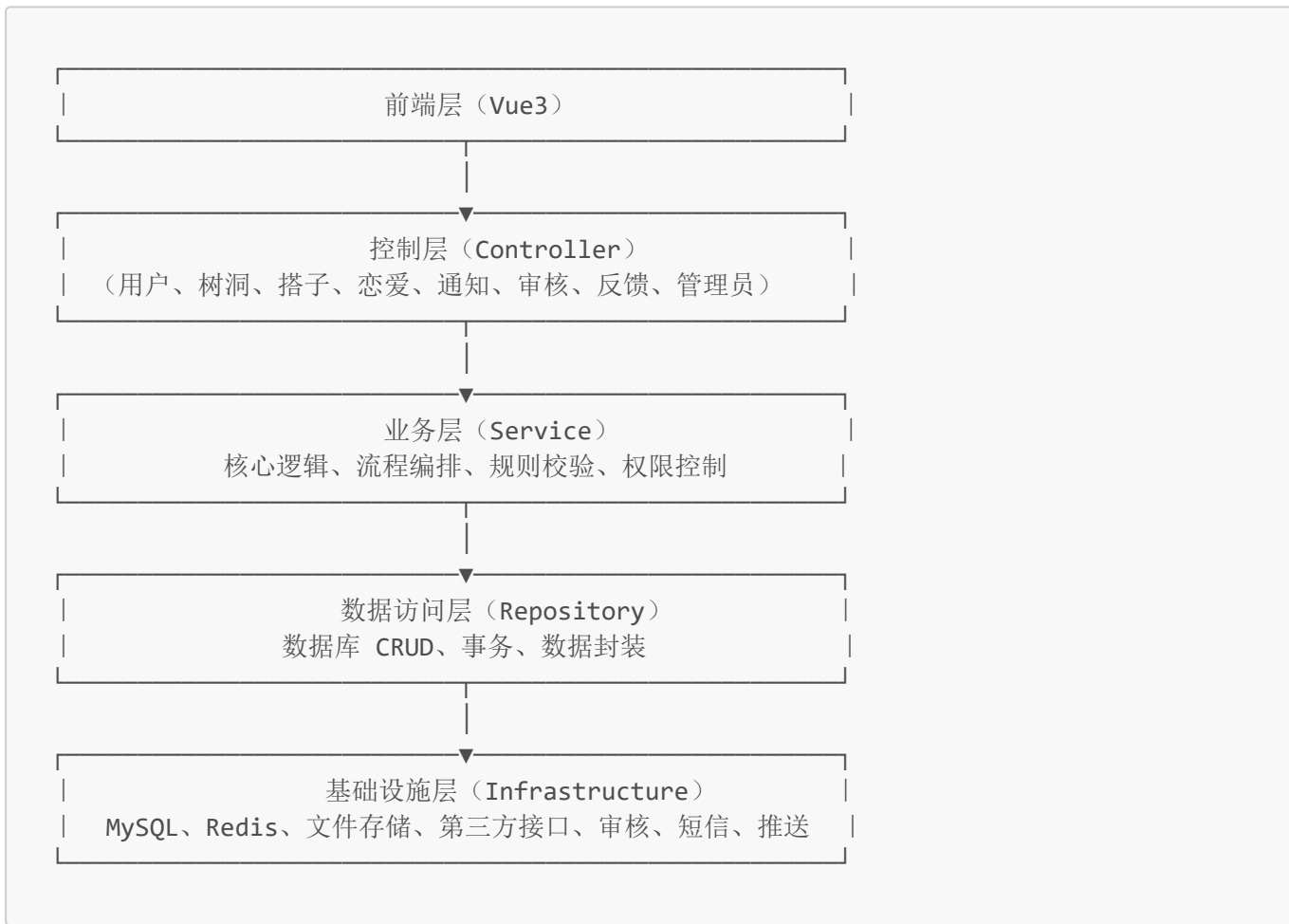
1.1 项目背景

本项目为**校园互助服务平台（青隅）**，面向高校大学生，提供树洞倾诉、找搭子、恋爱互助三大核心服务。团队规模：4 人 开发周期：10 周 最终架构：**分层架构 + MVC 表现层** 混合模式

1.2 架构定位

- 主体架构：**分层架构（Layered Architecture）**
- 表现层组织：**MVC 模式**
- 扩展机制：局部事件驱动（通知、日志）
- 暂不采用：微服务、分布式、事件驱动主架构

1.3 整体架构图



1.4 架构选择理由

- 符合 4 人团队 + 10 周周期，学习成本低、风险最低。
- 结构清晰、易于分工，模块边界明确。
- 完全支撑所有需求：用户、树洞、找搭子、恋爱互助、审核、通知、反馈、后台管理。
- 易开发、易调试、易测试，非常适合课程项目。

5. **性能足够**，本地调用无分布式开销，满足校园规模。
 6. **可维护性强**，层级分明，便于后续扩展。
-

2. 模块划分说明

2.1 总体模块划分

本平台共 **8 大核心业务模块**：

1. **用户模块 (User)**
 2. **树洞模块 (TreeHole)**
 3. **找搭子模块 (Partner)**
 4. **恋爱互助模块 (Love)**
 5. **消息通知模块 (Notify)**
 6. **内容审核模块 (Audit)**
 7. **管理员模块 (Admin)**
 8. **反馈建议模块 (Feedback)**
-

2.2 模块职责明细

(1) 用户模块

- 用户注册、登录、短信验证
- 学号认证、实名认证
- 个人资料、隐私设置、头像、兴趣标签
- 账号安全、密码找回、注销
- 提供用户信息查询接口给其他模块

(2) 树洞模块

- 匿名发布动态（文字 + 图片）
- 匿名浏览、点赞、评论、回复
- 分类、搜索、热度排序
- 动态管理（编辑、删除、可见范围）
- 内容审核接入
- 心理健康专栏展示

(3) 找搭子模块

- 发布搭子需求（学习/运动/吃饭/考研/出行等）
- 匹配条件：年级、专业、性别、时间、地点
- 自动匹配 + 主动申请
- 搭子聊天、群聊、评价体系
- 搭子关系管理（待确认/已建立/已结束）

(4) 恋爱互助模块

- 交友资料完善（照片、身高、星座、择偶要求）
- 交友需求发布、匹配、推荐
- 聊天、语音、礼物互动
- 交友安全提醒、违规举报
- 隐私保护、敏感信息隐藏

(5) 消息通知模块

- 系统公告、审核结果通知
- 点赞、评论、回复、匹配通知
- 消息免打扰、开关配置
- 多端同步

(6) 内容审核模块

- 文字违规词过滤
- 图片自动审核
- 人工审核工作台
- 审核记录、内容风险控制

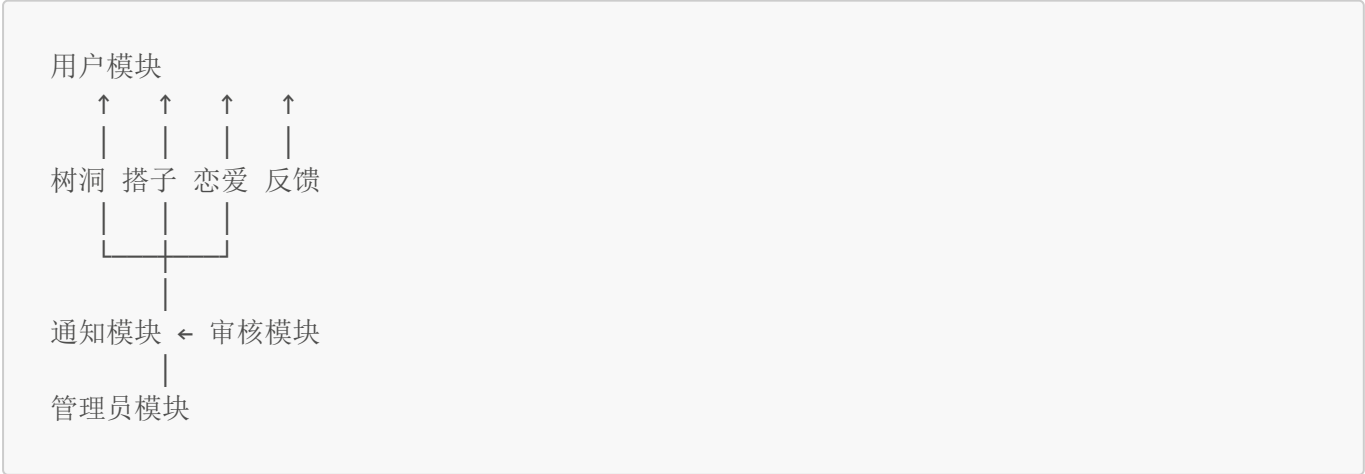
(7) 管理员模块

- 用户管理、封禁、解封
- 内容管理、删帖、置顶
- 违规词库管理
- 数据统计、日志、备份
- 反馈处理

(8) 反馈建议模块

- 用户提交反馈、上传截图
- 处理进度查询
- 管理员回复与记录

2.3 模块依赖关系



2.4 模块接口规范

- 调用方式：RESTful API + 本地方法调用
- 数据格式：JSON + DTO
- 统一返回格式：code / msg / data
- 所有跨模块调用必须通过接口，不直接依赖实现类

3. 技术选型说明

层次	技术选型	选择理由
前端框架	Vue 3	简单高效、生态成熟、适合快速开发
后端框架	Spring Boot	完美支持分层架构，注解开发、稳定易学
数据库	MySQL 8.0	轻量、稳定、易学，足够支撑校园规模
身份验证	JWT	无状态、轻量、适合前后端分离
文件存储	本地存储	简化部署、满足课程项目需求
图片审核	第三方审核接口	自动过滤违规内容，降低人工成本
短信服务	第三方短信接口	实现注册、登录、密码找回验证
部署方式	本地 JAR 包	环境简单、交付快速、10 周可完成

4. ADR 集合

ADR-001：采用分层架构 + MVC 表现层作为系统主架构

状态

已接受

背景

项目为校园互助服务平台（青隅），4 人团队、10 周周期，需支持用户、树洞、找搭子、恋爱互助、内容审核、消息通知等全业务功能，需从多种架构中选定主架构。

决策

我们决定采用**分层架构**为主体，**MVC 作为表现层组织方式**的混合架构。（diff）

理由

1. 开发复杂度低、学习成本低，适合小团队快速交付
2. 层级清晰、便于分工、调试简单、风险可控

- 3. 完全匹配 10 周周期，可保证按期稳定交付
- 4. 可完整支撑树洞、搭子、恋爱互助三大核心场景（diff）

后果（diff）

- 正面影响：开发效率高、结构清晰、易维护、便于答辩
- 负面影响：扩展性不如微服务，仅支持整体系统扩容
- 需要关注的风险：业务膨胀后可能出现逻辑耦合，需通过 DTO 与接口规范约束

AI 辅助记录

- **AI 初稿内容摘要：**推荐使用分层架构，理由为简单、稳定、易开发。
- **人工修订内容（diff）：**
 - **新增：**MVC 表现层定位
 - **新增：**混合架构模式
 - **新增：**结合树洞、搭子、恋爱互助业务场景
 - **新增：**质量属性（可维护性、安全性、性能）权衡
- **修订理由：**AI 输出过于通用，未结合项目实际业务与团队约束。

ADR-002：采用 MySQL 8.0 作为项目唯一数据库

状态

已接受

背景

平台需存储用户、树洞、搭子、恋爱交友、消息、评论、审核记录等数据，需确定数据库类型与数量。

决策

我们决定采用 **MySQL 8.0 单一数据库** 存储全部业务数据，不使用分库分表、NoSQL 或多数据源。（diff）

理由（diff）

- 1. 校园项目数据量小，单库完全满足需求
- 2. 团队学习MySQL成本较低
- 3. 无需处理分布式事务，架构简单稳定
- 4. 降低开发、测试、部署复杂度，保障 10 周可交付

后果

- 正面影响：架构简单、无分布式问题、开发快速稳定
- 负面影响：面对超大规模数据扩展性有限
- 需要关注的风险：表数量增多后可能变慢，将通过索引与分页优化

AI 辅助记录

- **AI 初稿内容摘要：**建议使用 MySQL，理由为流行、轻量、稳定。
 - **人工修订内容 (diff)：**
 - **明确拒绝：**多数据源、MongoDB 等 NoSQL 方案
 - **新增：**结合校园数据规模、周期紧张、团队技术约束进行论证
 - **修订理由：**AI 未体现工程决策中的取舍与约束，人工补充后更贴合项目实际。
-

ADR-003：采用前后端分离 + RESTful API + JWT 身份认证

状态

已接受

背景

平台支持 Web 端与移动端，需要统一接口、多端同步、安全登录、匿名机制与权限控制。

决策

我们决定采用**前后端分离架构**，基于 **RESTful API** 提供服务，并使用 **JWT** 实现身份认证与权限控制。(diff)

理由

1. 支持多端数据统一，便于未来扩展移动端
2. 接口清晰，前后端分工明确
3. JWT 无状态、轻量、易于部署 (diff)
4. 便于实现树洞匿名、登录权限、管理员权限 (diff)
5. 方案成熟、学习成本低、资料丰富

后果

- 正面影响：多端适配强、权限灵活、架构清晰
- 负面影响：JWT 无法主动销毁，必须设置合理过期时间
- 需要关注的风险：Token 泄露可能带来越权风险，需配合签名与 HTTPS (diff)

AI 辅助记录

- **AI 初稿内容摘要：**建议前后端分离与 Token 认证。
 - **人工修订内容 (diff)：**
 - **明确：**RESTful API 规范
 - **明确：**JWT 选型及其理由
 - **新增：**匿名机制、权限控制、安全风险分析
 - **修订理由：**AI 缺少工程实现细节，人工补充后更完整、可落地。
-

ADR-004：不采用微服务架构

状态

已废弃

背景

在架构选型中，对微服务架构进行了可行性评估。微服务具备良好扩展性与模块隔离能力，但需结合团队规模、周期与业务场景进行权衡。

决策

我们决定**不采用微服务架构**，维持单体分层架构方案。（diff）

理由

- 1. 微服务需要引入网关、注册发现、分布式事务等额外技术体系，实施成本高
- 2. 项目业务边界清晰，单体架构已能满足维护性与稳定性需求
- 3. 4 人团队难以承担微服务运维、联调、故障排查成本（diff）
- 4. 10 周周期内完成微服务交付风险较高（diff）
- 5. 校园用户规模与并发量较低，无需微服务级别的扩展能力（diff）

后果

- 正面影响：架构简洁、开发高效、交付可控、学习成本低
- 负面影响：不支持模块级独立扩容
- 需要关注的风险：未来业务增长后可能需要重构

AI 辅助记录

- **AI 初稿内容摘要：**中立介绍微服务优缺点，未给出明确取舍。
- **人工修订内容（diff）：**
 - **明确废弃**微服务架构
 - **新增：**结合团队规模、开发周期、业务规模进行论证
- **修订理由：**AI 未结合实际工程约束，人工修订使决策更合理、可落地。

ADR-005：是否引入 Redis 做热点数据缓存

状态

待讨论

背景

树洞首页、热门搭子、用户会话可使用缓存提升性能，但会增加架构与部署复杂度。

决策

我们决定**现阶段暂不引入 Redis**，以最简架构开发核心功能，后期根据性能测试结果再决定是否接入。（diff）

理由

1. 初期用户量少，MySQL 足以支撑业务
2. 减少外部组件，降低部署与调试成本
3. 不影响核心功能交付
4. 后期可低代价接入

后果

- 正面影响：架构极简、开发更快、交付风险更低（diff）
- 负面影响：高并发场景下性能可能不足
- 需要关注的风险：若活跃用户增加，接口响应可能变慢

AI 辅助记录

- **AI 初稿内容摘要**：建议直接引入 Redis 提升性能。
 - **人工修订内容（diff）**：
 - **调整**：改为“暂不引入、后期评估”
 - **新增**：不过早优化，符合课程项目开发节奏
 - **修订理由**：AI 偏向技术优化，人工修订更贴合项目实际阶段目标。
-

5. 质量属性设计

- 性能：接口响应 < 300ms，支持校内规模并发
 - 可用性：统一异常处理，界面友好提示
 - 安全性：密码加密、JWT 认证、内容审核、匿名保护、信息脱敏
 - 可维护性：分层清晰、模块独立、DTO 隔离
 - 可扩展性：外部依赖面向接口，便于替换与升级
-

6. 架构优缺点总结

优点

- 开发简单，10 周可交付
- 学习成本低，适合 4 人团队
- 模块清晰、易分工
- 稳定、不易出BUG
- 安全、匿名、审核机制完善

缺点

- 无法独立扩容模块
 - 后期业务变大会略显臃肿
-